

LLM ENGINEERING

Agentic Dependency Parsing

in Low-Resource Settings

Jan Raring, Dang Nhat Quang, Nikhila Gadge, Han Dai

What is & Why should we care?

What are dependency Trees

Dependency parsing builds a tree structure for a sentence

Resolves ambiguities in language processing

It shows how words are syntactically related by connecting each word to its head `

Applications:

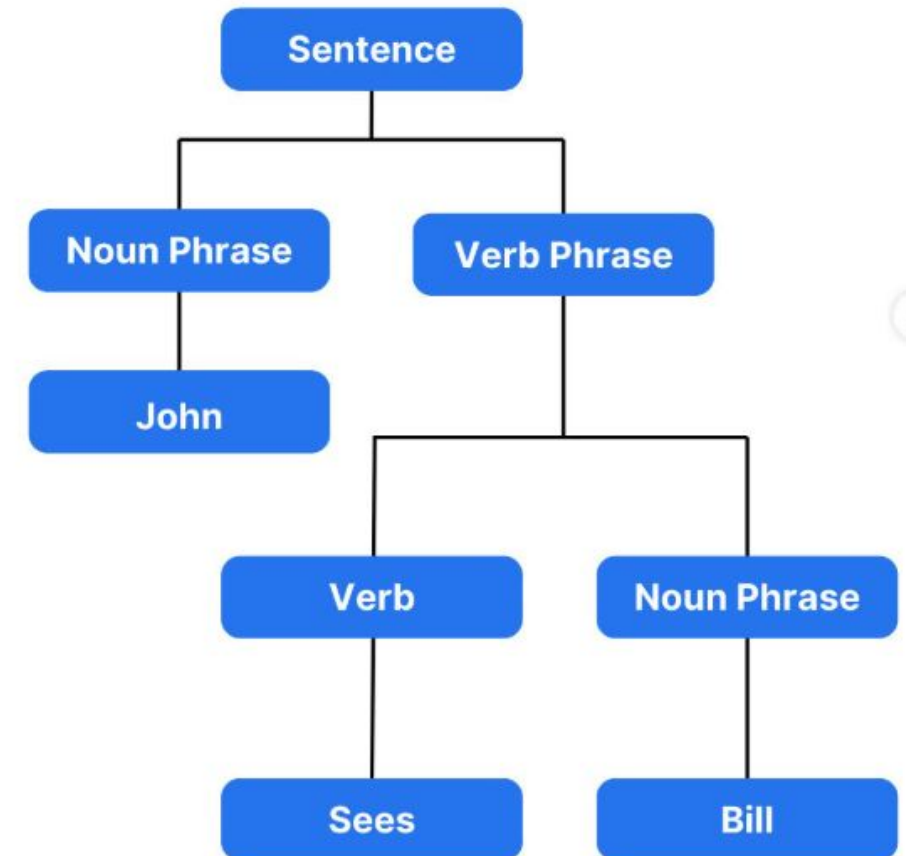
machine translation

Sentiment Analysis

information extraction

Supports complex Applications: Q-A & Voice recognition systems

Dependency Parsing in Machine Learning



Introduction

The Bottleneck of Syntax Research

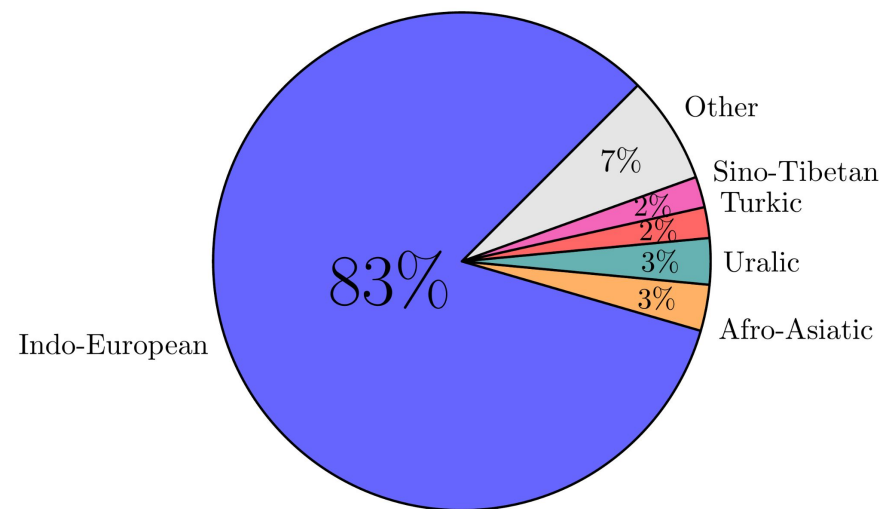
Treebanks are essential, but building them is extremely EXPENSIVE!!

Dependency parsing builds a tree of head–dependent relations for every sentence, forming the indispensable backbone of syntax, grammar, and typology research

The Dilemma:

Manual annotation of syntax trees is slow.

While high-resource languages leverage semi-automatic annotation tools, low-resource languages are left with nothing but raw manual labor



The Resource Gap in Modern NLP

High-Resource Languages

- **Ready-to-use Tools:**
Stanza, spaCy, and NLTK provide highly robust pre-trained supervised parsers
- **Plentiful Data:**
Thousands of sentences of gold-standard labeled treebanks already exist
- **Accelerated Growth:**
Pre-annotation speeds up further scaling effortlessly

Low-Resource Languages

- **No Existing Parsers:**
0 trained parsers exist for minor or regional dialects
- **Expert Bottleneck:**
Treebank construction relies completely on slow, expert linguistic labor

Our Idea:

Skip training purpose-built neural nets entirely.

Can we equip a general-purpose LLM with the right tools to parse directly?

Research Questions

Can an LLM agent, equipped with adequate tools, function as a general-purpose dependency parser
a.k.a. eliminating the need to train purpose-built neural networks for every new language?

If the agent's tools (validity checking, retrieval, morphology, valency) generalize across languages, the same recipe could bootstrap treebanks anywhere UD-style data is scarce

Related Work



Neural dependency parsers

Stanza, spaCy, and NLTK train supervised models per language, accurate, but only where labeled treebanks already exist



Cross-lingual transfer

García-Ferrero transfers parsing knowledge from high to low-resource languages via shared representations, without training data in the target language



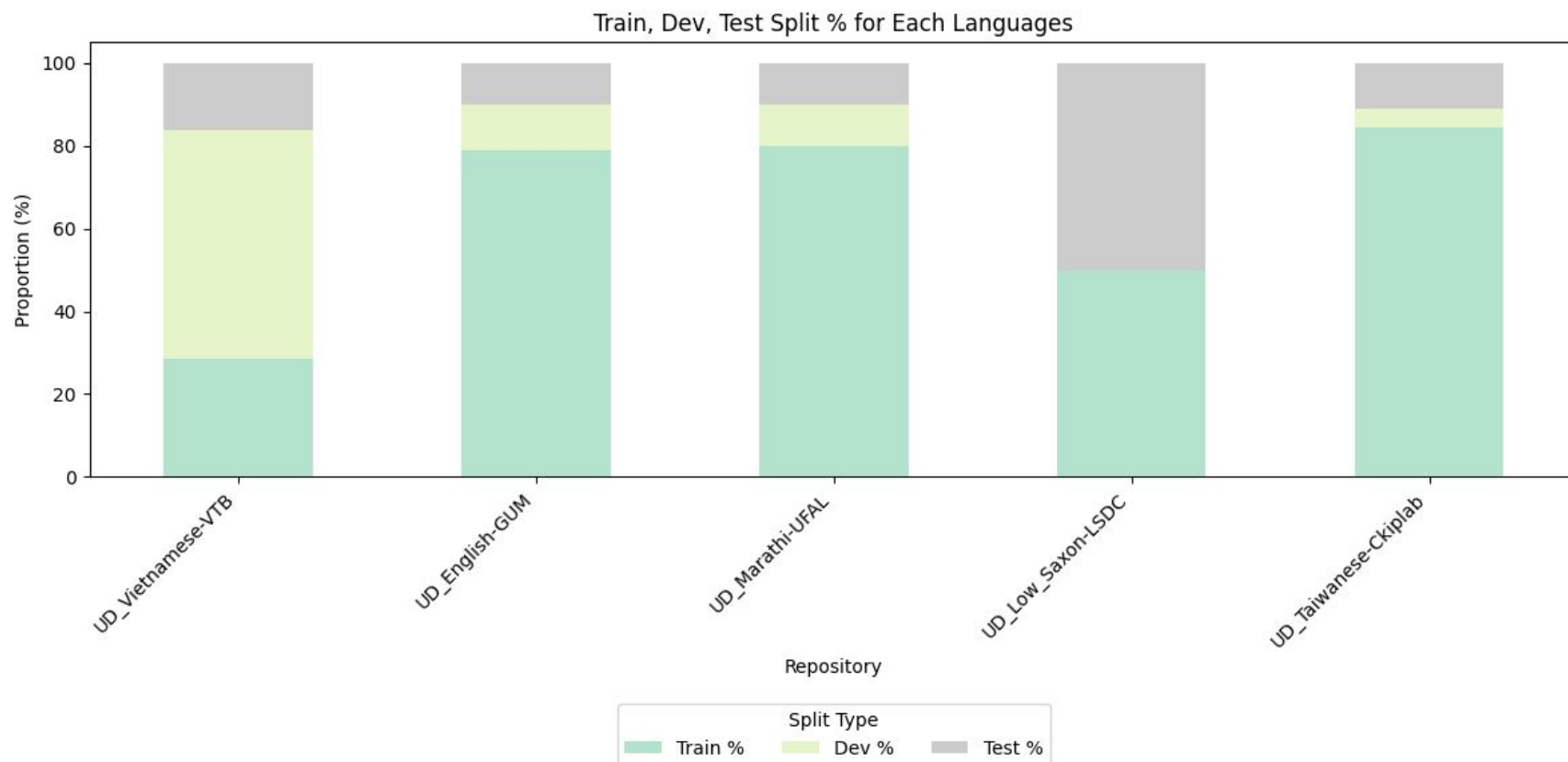
Our gap to fill

No prior work tests whether an off-the-shelf LLM agent with retrieval, validity-checking, and lexical tools, and whether they can parse a language directly, with zero trained parser

Languages

5 Universal Dependencies treebanks:

- Low Saxon
- Marathi
- Vietnamese
- Taiwanese
- English



Project Pipeline

0 Data Audit

Download & verify 4-5 UD treebanks, count sentences/tokens, and measure non-projectivity rates

1 Baselines

- Direct prompting
- No tools
- Measure UAS and parsing success

2 Agentic Parsing

Build the tool-calling harness:

-  Morphology lookup
-  Sentence retrieval
-  BoW retrieval
-  Tree validation

3 Without Tools

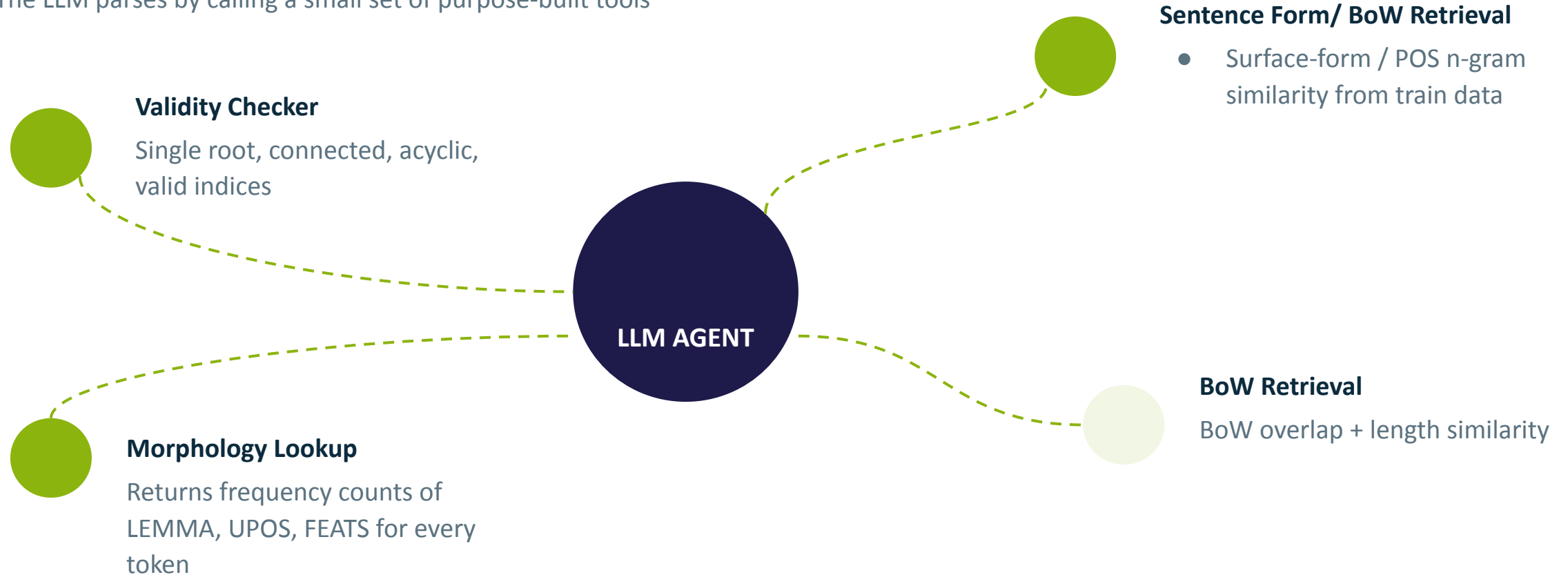
The NLP pipeline was run without tool calling access

4 Evaluation

Unlabeled Attachment Score
Parsing rate
Number of tokens generated
Number of tools called per input

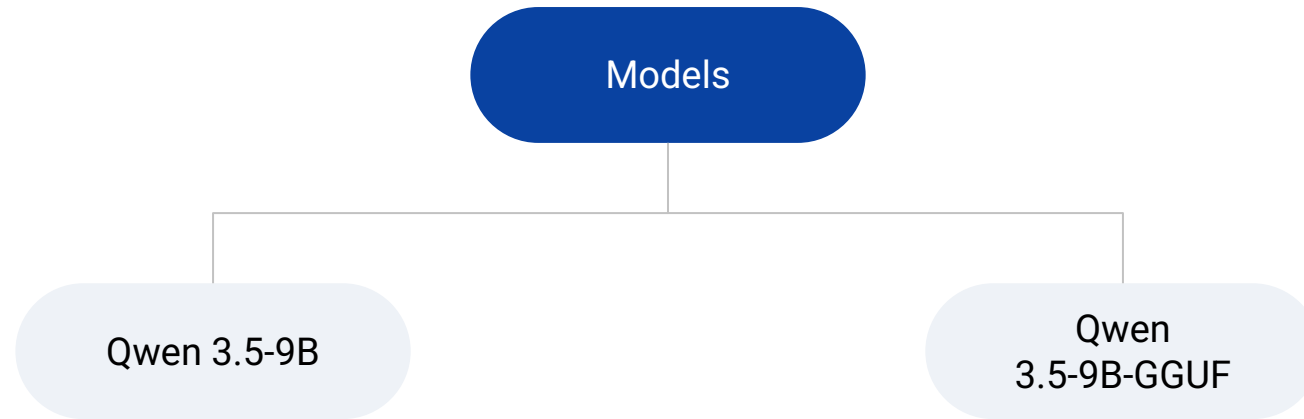
Agentic Architecture

The LLM parses by calling a small set of purpose-built tools



Methodology

LLM Models



Model Card: Qwen/Qwen3.5-9B
Quantization: BitsAndBytes 4-bit NF4
Architecture: Transformers
Hardware: NVIDIA A100 GPU

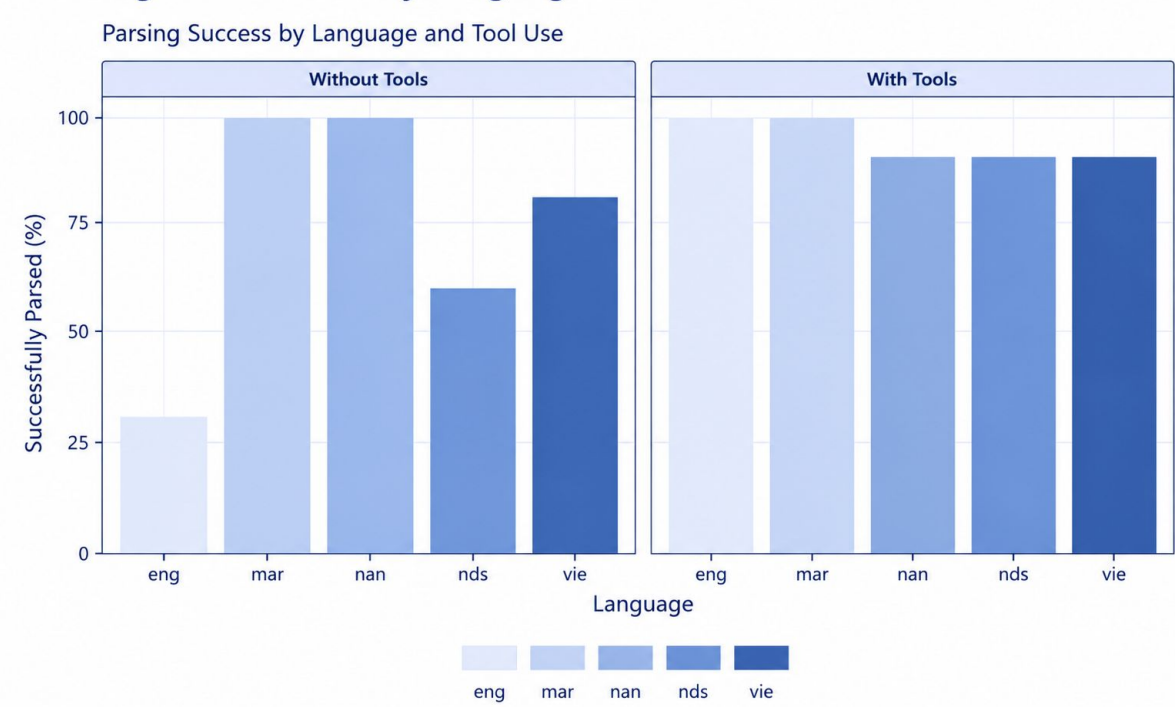
Model Card: unsloth/Qwen3.5-9B-GGUF
Quantization: GGUF Q4_K_M
Architecture: llama.cpp
Hardware: Intel Core Ultra 7 H155

Results

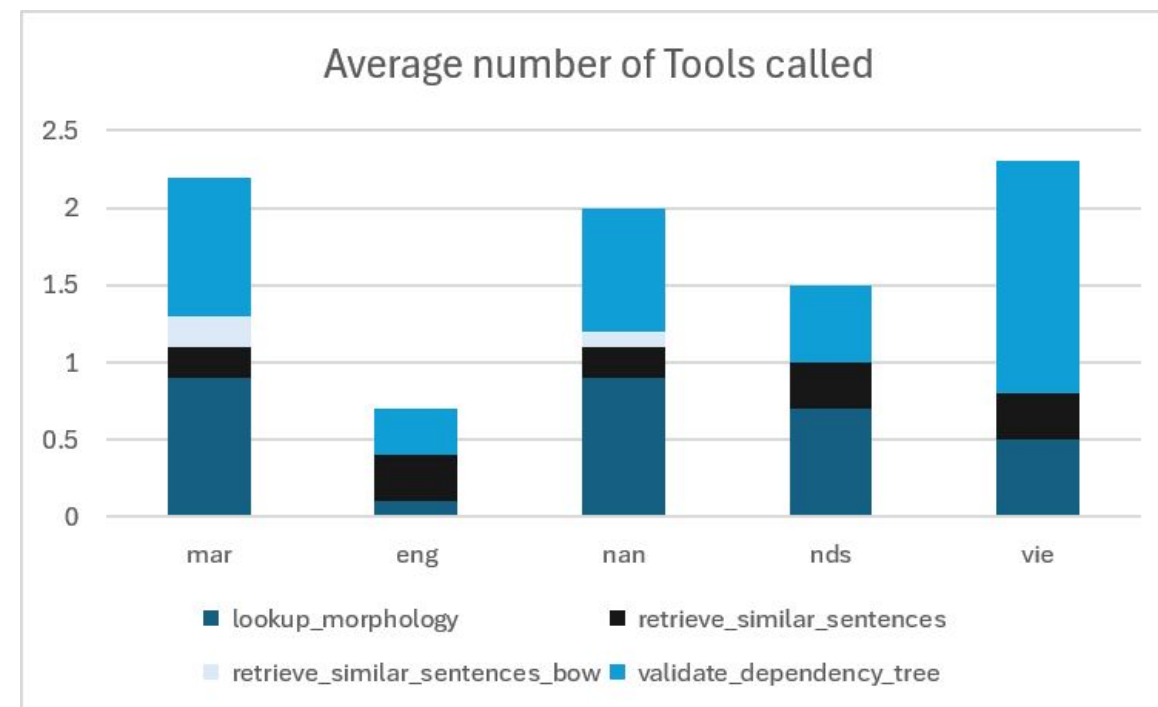
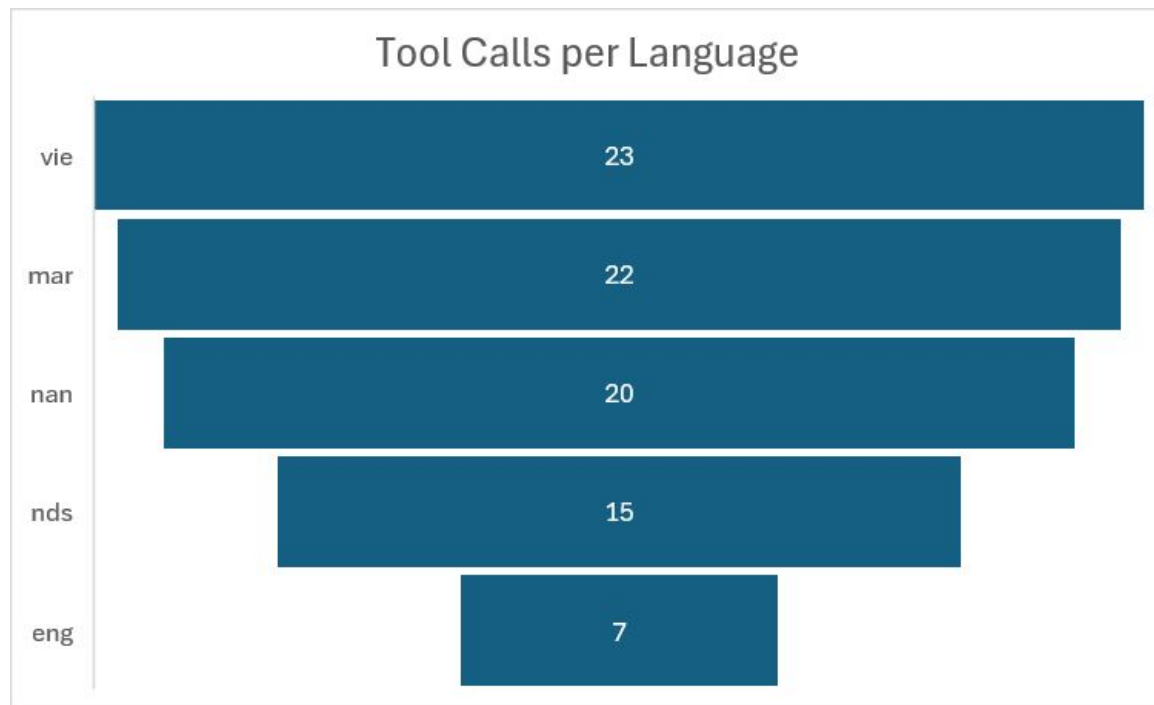
Tools turn scarce evidence into structured support

- On an average the tools support increased the Unlabeled Attachment Score by +3%
- On an average the Parsing success improved with the tools
- English benefited the most with +15% UAS score increase

Parsing Success Rate by Language and Tool Use



Results



- Most tools were called for Vietnamese while English called the least number of tools
- Validate Dependencies tool was called the most number of times
- Morphology Statistics was called the most for lower resource languages

Conclusion

- On average, having tools at its disposal, the model used much fewer tokens to give a final output, thereby increasing the chance of producing any output at all.
- The downside to the faster output generation seems to be increased sloppiness when it comes to generating structurally valid trees.
- Considering English and Low Saxon, seemingly contradictory to the above observations, it looks like having the option to use auxiliary tools noticeably increased the accuracy on longer, more complicated sentences.

Discussion

Limitations and Future Scope

- Ablation Experiment could be conducted to know the performance and preference of tools individually
- Experimentation with more tools and models with greater multilingual support
- Prompt Engineering and Hyperparameters adjustment like Temperature, Top_p, max_new_tokens

Thank you for your attention

Questions & discussion welcome